

Outline of First Degree Thesis

Contents

1	Aim and Objectives	2
1.1	Proposed Title	2
1.2	Research Question	2
1.3	Aim	2
1.4	Objectives	2
2	Background	2
2.1	Sudoku as an Exactly Verifiable Constraint-Satisfaction Problem	2
2.2	Representation Invariance	3
2.3	LLM Reasoning on Formal Tasks	4
2.4	Arbitrary Binding, Semantics, Tokenization, and Unicode	4
2.5	Research Gap and Claim Boundaries	4
3	Plan of Work	5
3.1	Model Cohort Used Throughout	5
3.2	Experiment 1	5
3.2.1	Output	5
3.2.2	Procedure	5
3.2.3	Example Solution	8
3.3	Experiment 2	8
3.3.1	Output	8
3.3.2	Procedure	8
3.4	Experiment 3	9
3.4.1	Output	9
3.4.2	Procedure	9
3.5	Experiment 4	9
3.5.1	Test	9
3.5.2	Procedure	9
3.6	Experiment 5	10
3.6.1	Test	10
3.6.2	Procedure	10
3.7	Experiment 6	10
3.7.1	Test	10
3.7.2	Procedure	10
3.7.3	Example Solution	11
3.8	Experiment 7	11
3.8.1	Test	11
3.8.2	Procedure	11
3.8.3	Example Solution	11
3.9	Experiment 8	11

3.9.1	Test	11
3.9.2	Procedure	12
3.10	Experiment 9	12
3.10.1	Test	12
3.10.2	Procedure	12
3.10.3	Example Solution	12
3.11	Experiment 10	12
3.11.1	Test	12
3.11.2	Procedure	12

4 Bibliography and References 13

1 Aim and Objectives

1.1 Proposed Title

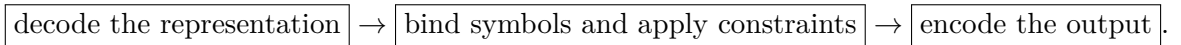
Beyond Arabic Digits: Probing Large Language Model Logical Reasoning Under Symbol-Set Variation

1.2 Research Question

When a large language model (LLM) can solve a Sudoku puzzle, does it preserve that ability when Arabic numerals are replaced with a different bijective symbol system? If accuracy changes, can the change be attributed to input decoding, symbol-to-value binding, constraint reasoning, output encoding, tokenization, Unicode handling, or formatting?

1.3 Aim

The aim is to measure representation invariance in LLM Sudoku solving and localize representation-induced failures within



1.4 Objectives

1. Construct and certify 300 uniquely solvable 9×9 Sudoku puzzles: 100 solver-certified easy, 100 medium, and 100 hard puzzles.
2. Evaluate identical puzzles under nine bijective symbol systems while holding clues, rules, prompts, output format, and inference settings constant.
3. Separate decoding, arbitrary binding, constraint reasoning, and generation through input/output crosses, translation controls, canonicalization, random mappings, and semantic-conflict tests.
4. Quantify tokenization, Unicode, verification, repair, and response-format effects with paired experiments.
5. Produce a reproducible evaluator and statistical analysis that distinguish logical errors from parser, transport, API, and formatting failures.

2 Background

2.1 Sudoku as an Exactly Verifiable Constraint-Satisfaction Problem

Let

$$G = \{g_{r,c} \mid r, c \in \{1, \dots, 9\}\}$$

be the 81 cells and let $D = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}$ be the value domain. A completed grid is correct only if it preserves every clue and, for every row r , column c , and box b ,

$$\{g_{r,1}, \dots, g_{r,9}\} = D,$$

$$\{g_{1,c}, \dots, g_{9,c}\} = D,$$

and

$$\{g_{r,c} : (r, c) \in b\} = D.$$

The evaluator can therefore check all 81 cells, all clues, nine rows, nine columns, and nine boxes without subjective grading.

Difficulty cannot be inferred from clue count alone. Two 30-clue puzzles can differ because one uses only naked and hidden singles whereas another requires a hidden pair, box-line reduction, and dependent deductions. Pelánek likewise finds that technique complexity and dependency structure contribute to difficulty [1].

The geometry remains fixed:

```
grid:          9×9
cells:         81
boxes:         nine 3×3 boxes
abstract values: 9
default empty marker: .
```

2.2 Representation Invariance

Define the abstract values as $V = \{V_1, V_2, \dots, V_9\}$. Each visible alphabet $S = \{s_1, \dots, s_9\}$ is introduced by a bijection

$$\sigma : V \leftrightarrow S.$$

For Greek letters:

```
V1→α V2→β V3→γ V4→δ V5→ε
V6→ζ V7→η V8→θ V9→ι
```

Thus $5\ 3\ \dots\ 7\ \dots$ and $\epsilon\ \gamma\ \dots\ \eta\ \dots$ are isomorphic. If f is the model-and-parser pipeline, exact invariance requires

$$\sigma^{-1}(f(\sigma(p))) = f(p)$$

whenever both sides return completed grids.

InvariantBench evaluates consistency under semantics-preserving transformations [5], while LGMT derives metamorphic tests from logical equivalences [10]. Consistency is necessary but insufficient: a model can repeat the same wrong answer. Sudoku adds independent constraint

checking. GSM-Symbolic reports performance variation across controlled mathematical variants [6], but tokenization, numerical difficulty, and learned associations can accompany surface changes.

2.3 LLM Reasoning on Formal Tasks

Sudoku-Bench shows that full-grid solve rates decline as Sudoku tasks become more complex [2], but its variants can change rules as well as representation. This thesis changes only value labels while preserving rules, clues, and the unique solution.

Shah et al. show that models trained on solver traces can learn search and constraint propagation for Sudoku and Zebra puzzles [3]. Poor zero-shot performance therefore does not establish an architectural impossibility. Work on graph colouring also shows that fluent self-criticism is not exact verification [4]. A model statement such as `the grid is valid` will never override the deterministic evaluator.

2.4 Arbitrary Binding, Semantics, Tokenization, and Unicode

Symbol tuning improves adherence to arbitrary mappings such as `positive→foo` [7]. This result shows that arbitrary binding is learnable, but does not establish reliable generation of a consistently mapped 81-cell grid.

Yan et al. report failures when digits in elementary addition are bijectively renamed [11]. Such failure does not separate memorization, tokenization, mapping mistakes, output generation, or working-memory demand. The experiments below isolate these alternatives.

Semantic interference is tested with

ordinary: `V1→1, V2→2, ..., V9→9`
 conflicting: `V1→9, V2→4, ..., V9→2`
 neutral: `V1→KAV, V2→MIP, ..., V9→WIF`

Worse performance for conflicting familiar labels than token-matched neutral labels supports semantic interference, but a globally consistent wrong permutation must still be excluded.

Tokenizer choice and vocabulary coverage affect multilingual performance [8], [9]. One visible symbol may contain several Unicode code points, bytes, or tokens. Each symbol-model pair will therefore record:

Unicode code-point sequence
 UTF-8 byte count
 grapheme count
 token count in isolation
 token count after whitespace
 token count inside a row
 total prompt-token count

Local tokenizers provide exact counts and token IDs for open weights. The two commercial APIs provide request-level token counting; token IDs are consequently an open-weight-only diagnostic, not a cross-model dependent variable.

2.5 Research Gap and Claim Boundaries

Existing work does not locate where a value-renaming failure occurs in 9×9 Sudoku after Unicode, token length, decoding, output alphabet, formatting, difficulty, and inference resources

are controlled. The main benchmark alone can establish sensitivity to logically irrelevant re-naming. A claim about representation-dependent constraint reasoning requires a residual gap after the mechanism controls. The conclusions will remain specific to 9×9 Sudoku and will not be generalized to all constraint-satisfaction, mathematical, or real-world reasoning tasks.

3 Plan of Work

3.1 Model Cohort Used Throughout

The following candidate editions were current on 18 August 2026. **Exact LLMs I will fix after considering the compute capacity of GPUs we have.**

Access	Exact model
open weights	Qwen/Qwen3.8-27B [12]
open weights	zai-org/GLM-5.2 [13]
open weights	moonshotai/Kimi-K3 [14]
API	gpt-5.6-terra [15]
API	claude-sonnet-5 [16]

The nine main families are:

Symbol family	Ordered symbols representing V_1 to V_9
Arabic digits	1 2 3 4 5 6 7 8 9
Devanagari numerals	१ २ ३ ४ ५ ६ ७ ८ ९
Bengali numerals	১ ২ ৩ ৪ ৫ ৬ ৭ ৮ ৯
Uppercase Latin	A B C D E F G H I
Lowercase Latin	a b c d e f g h i
Greek letters	α β γ δ ε ζ η θ ι
Abstract symbols	△ □ ○ ☆ × † ‡ § ¶
Emoji	🔴 🟡 🟢 🟠 🟣 ⚫ ⚪
Nonce labels	KAV MIP ZOT RUL BEK DAX PEV NUG WIF

3.2 Experiment 1

3.2.1 Output

A solver-certified dataset of 300 uniquely solvable Sudoku puzzles, together with a validated evaluator, parser, Unicode pipeline, and model qualification record. These outputs prevent puzzle, software, compute, and transport defects from being mistaken for model failures.

3.2.2 Procedure

For every item, generate a complete solution, remove clues one at a time, and use an exact counter that stops after finding two solutions. Retain a removal only if exactly one solution remains. Store the puzzle, solution, clue mask, seed, generator version, and solver certificate. The final dataset is

$$100 \text{ easy} + 100 \text{ medium} + 100 \text{ hard} = 300.$$

The evaluator will return one overall outcome and a list containing results. The overall outcome is CORRECT, INCORRECT, or NOT_EVALUATED. NOT_EVALUATED is used only when no candidate grid is available because of an operational result such as API_ERROR or TIMEOUT.

For example, a response that changes a clue and violates both a row and a box will be recorded as

```
{
  "outcome": "INCORRECT",
  "results": [
    {"type": "GIVEN_MODIFIED", "cell": "r1c1",
     "expected": "5", "actual": "6"},
    {"type": "ROW_CONSTRAINT_ERROR", "row": 1,
     "duplicate": "6", "missing": "5"},
    {"type": "BOX_CONSTRAINT_ERROR", "box": 1,
     "duplicate": "6", "missing": "5"}
  ]
}
```

The labels for results are:

Label	Meaning and example
CORRECT	The response contains exactly 81 permitted values, preserves every clue, and satisfies all row, column, and 3×3 box constraints.
GIVEN_MODIFIED	At least one original clue was changed. For example, if the puzzle specifies $r1c1 = 5$ but the response contains $r1c1 = 6$, this label applies even if the remaining grid is valid.
ROW_CONSTRAINT_ERROR	At least one row does not contain all nine values exactly once. For example, row 4 contains two ϵ symbols and no θ . The same response may also receive column or box errors.
COLUMN_CONSTRAINT_ERROR	At least one column does not contain all nine values exactly once. For example, column 7 contains two 4 values and no 9. The same response may also receive row or box errors.
BOX_CONSTRAINT_ERROR	At least one 3×3 box does not contain all nine values exactly once. For example, the upper-left box contains two β symbols and no η . The same response may also receive row or column errors.
INVALID_SYMBOL	The response contains a value outside the declared alphabet. For a Greek alphabet $\{\alpha, \beta, \gamma, \delta, \epsilon, \zeta, \eta, \theta, \iota\}$, an output cell containing κ is invalid.
GLOBAL_BINDING_ERROR	The grid becomes the intended abstract solution after applying one consistent permutation to every output symbol. For example, all intended α values are returned as β and all intended β values as α .

Label	Meaning and example
LOCAL_MAPPING_ERROR	Symbol mapping is inconsistent between cells, so no single global permutation explains the response. For example, one intended α is returned as β , while another intended α is returned correctly as α .
WRONG_CELL_COUNT	After parsing, the response contains a number of values other than 81. For example, eight complete rows and one row with eight values produce 80 cells.
WRONG_ROW_COUNT	The requested row-based format contains a number of rows other than nine. For example, the response contains eight lines with nine values each.
FORMAT_ERROR	The response is not exactly a 9×9 Sudoku grid in the format required by that experiment. For example, a row uses commas when the required separator is a space.
UNICODE_ERROR	Unicode decoding, normalization, or grapheme segmentation fails. For example, a stored symbol is corrupted during UTF-8 decoding or an emoji variation sequence is split into separate cells.
NO_FINAL_ANSWER	The model provides discussion or reasoning but no completed grid that the evaluator can extract.
TRUNCATED_OUTPUT	Generation ends because of an output limit or transport cutoff before the completed grid is returned. For example, only rows 1–6 are present and the finish reason reports a length limit.
REFUSAL	The model explicitly declines to solve the task instead of returning a candidate grid.
API_ERROR	The provider returns an unsuccessful request after the fixed retry policy, such as an HTTP 500 response after five attempts. This is an operational failure, not a logical error.
TIMEOUT	The request exceeds the preregistered time limit without a completed response. This is reported separately from model-answer accuracy.

The parser reads only the Sudoku grid in the exact format required by the experiment. In the main experiments, a valid response contains exactly nine lines, and each line contains exactly nine space-separated symbols. Experiments that compare output formats use a separate exact parser for each specified grid format.

Unicode tests verify that every permitted symbol is read as one cell and survives UTF-8 storage without alteration. This is necessary for alphabets containing symbols such as α , $\aleph$, or $\bullet$. The required invariant is

$$\text{decode}(\text{encode}(p, \sigma), \sigma^{-1}) = p.$$

Each model receives five qualification requests spanning Arabic, Greek, emoji, nonce, and Devanagari. Open-weight logs include hashes, engine, GPU, precision, and batch size; API logs include exact model ID, access date, response metadata, and accepted reasoning controls. Failure to qualify any model stops protocol freeze rather than causing silent substitution.

3.2.3 Example Solution

The exact solver confirms one solution for

```
5 3 . . 7 . . . .
6 . . 1 9 5 . . .
. 9 8 . . . . 6 .
8 . . . 6 . . . 3
4 . . 8 . 3 . . 1
7 . . . 2 . . . 6
. 6 . . . . 2 8 .
. . . 4 1 9 . . 5
. . . . 8 . . 7 9
```

Its unique solution is

```
5 3 4 6 7 8 9 1 2
6 7 2 1 9 5 3 4 8
1 9 8 3 4 2 5 6 7
8 5 9 7 6 1 4 2 3
4 2 6 8 5 3 7 9 1
7 1 3 9 2 4 8 5 6
9 6 1 5 3 7 2 8 4
2 8 7 4 1 9 6 3 5
3 4 5 2 8 6 1 7 9
```

3.3 Experiment 2

3.3.1 Output

A calibrated difficulty set, qualification results for every model, and fixed output limits for the main experiments. The selected puzzles must avoid accuracy floors and ceilings.

3.3.2 Procedure

Easy puzzles use only naked and hidden singles. Medium puzzles require a naked pair, hidden pair, pointing pair, or box-line reduction. Hard puzzles require longer propagation, an advanced technique, or controlled search. A certificate can be

```
{
  "puzzle_id": "H037",
  "difficulty": "hard",
  "deduction_steps": 84,
  "techniques": ["hidden_single", "naked_pair", "box_line"],
  "backtracking_required": true,
  "maximum_search_depth": 3,
```

```
"search_nodes": 47
}
```

The pilot uses 20 puzzles per tier, four representations (Arabic, uppercase Latin, Greek, and emoji), and all five models:

$$60 \times 4 \times 5 = 1,200 \text{ calls.}$$

Target Arabic ranges are 80–95% easy, 40–70% medium, and 10–40% hard. **Hard: 0/100** is unusable; **Arabic: 32/100**, **Greek: 19/100**, and **emoji: 8/100** retains measurable variation. Pilot data are excluded from confirmatory results if they affect any protocol choice.

3.4 Experiment 3

3.4.1 Output

A representation-cost registry and a frozen confirmatory protocol. The registry contains token, byte, code-point, and grapheme measurements for every symbol-model pair; the protocol records all experimental decisions before confirmatory outcomes are observed.

3.4.2 Procedure

For every symbol-model pair, record code points, UTF-8 bytes, graphemes, token counts in isolation, after whitespace, and in a row, plus total prompt tokens. Open-weight models additionally expose token IDs. Mean clue-token cost is

$$T_s = \frac{\text{tokens encoding visible clues under } s}{\text{number of visible clues}}.$$

Freeze the 300 IDs, certificates, alphabets, mappings, prompts, output format, model IDs, commits, hashes, reasoning and sampling settings, retry/exclusion rules, evaluator version, hypotheses, metrics, and tests. A model replacement is allowed only under a preregistered rule before any main inference and without inspecting accuracy.

3.5 Experiment 4

3.5.1 Test

To test whether changing only visible value labels changes complete-grid accuracy.

3.5.2 Procedure

Every model receives every puzzle under all nine families:

$$300 \times 9 \times 5 = 13,500 \text{ primary calls.}$$

The standard transformed prompt is

Solve the following 9×9 Sudoku.

Valid symbols:

α β γ δ ε ζ η θ ι

Rules:

- Every row must contain each valid symbol exactly once.
- Every column must contain each valid symbol exactly once.
- Every 3×3 box must contain each valid symbol exactly once.
- Do not change the given cells.

Puzzle:

```

ε γ . . η . . . .
ζ . . α ι ε . . .
. ι θ . . . . ζ .
...

```

Return only 9 lines of 9 space-separated symbols.

Only the alphabet declaration and visible values change. Puzzle, clues, difficulty, wording, output format, effort, limits, decoding, retries, parser, and evaluator remain fixed.

For family s ,

$$A_s = \frac{\text{\#correct completed responses under } s}{\text{\#valid completed calls under } s}.$$

Report counts, percentages, and 95% intervals, for example $178/300 = 59.3\%$. API_ERROR and TIMEOUT are reported separately.

3.6 Experiment 5

3.6.1 Test

To test whether renaming breaks puzzles that the same model solves in Arabic and to analyse paired outcomes correctly.

3.6.2 Procedure

For each model define $B = \{p : \text{Arabic solution is correct}\}$. Then

$$R_s = \frac{|\{p \in B : p \text{ is correct under } s\}|}{|B|}, \quad F_s = 1 - R_s.$$

If Arabic is correct on 210 puzzles and Greek on 142 of those, $R_{\text{Greek}} = 142/210 = 67.6\%$ and $F_{\text{Greek}} = 32.4\%$.

Report R_s and F_s by model, symbol family, and difficulty. Also report cell errors, constraint labels, binding errors, and termination failures for each group.

3.7 Experiment 6

3.7.1 Test

To separate transformed-input decoding from transformed-output generation.

3.7.2 Procedure

Use the same 60-puzzle subset on all models:

Condition	Input	Output	Isolated requirement
A	Arabic	Arabic	baseline
B	Greek	Greek	decoding, reasoning, Greek output
C	Greek	Arabic	decoding and reasoning
D	Arabic	Greek	Greek output

Condition C gives $\alpha \rightarrow 1$, $\beta \rightarrow 2$, ..., $\iota \rightarrow 9$. If C approaches A but B does not, output encoding is implicated. If C remains below A, decoding or binding remains impaired. No-Sudoku controls ask for copying, mapping translation, coordinate retrieval, occurrence counting, and conversion of a supplied correct grid.

3.7.3 Example Solution

Input row: 5 3 4 6 7 8 9 1 2

Greek row: ϵ γ δ ζ η θ ι α β

Grid row: ϵ γ . . η

Question: Which symbol is in column 5?

Answer: η

Sequence: ϵ γ η

Answer: 5 3 7

3.8 Experiment 7

3.8.1 Test

To test TokensPerSymbol $\uparrow \Rightarrow$ Accuracy $\downarrow$ while separating token length from visible length and Unicode complexity.

3.8.2 Procedure

For each model, construct neutral alphabets whose nine labels each require exactly one, two, or three tokens. Verify each label alone, after whitespace, and inside a complete row. All models receive the same 60 puzzle IDs, although label strings are tokenizer-specific. A second comparison token-matches labels while varying grapheme and byte complexity. Regress correctness on prompt tokens, mean clue tokens, bytes, and graphemes. A residual family effect argues only that token count is incomplete, not irrelevant.

3.8.3 Example Solution

Set A: nine labels verified as 1 token each

Set B: nine labels verified as 2 tokens each

Set C: nine labels verified as 3 tokens each

3.9 Experiment 8

3.9.1 Test

To distinguish arbitrary-binding sensitivity from alphabet familiarity and conflict with learned symbol meanings.

3.9.2 Procedure

All models receive 60 puzzles under the standard uppercase mapping and five seed-fixed random mappings:

standard: V1→A V2→B V3→C V4→D V5→E V6→F V7→G V8→H V9→I
random: V1→F V2→A V3→I V4→C V5→H V6→D V7→B V8→G V9→E

Also compare ordinary digits, permuted digits, ordinary number words, conflicting number words, and token-matched nonce labels. Variance across random mappings estimates binding sensitivity; a conflict penalty relative to neutral labels supports semantic interference.

3.10 Experiment 9

3.10.1 Test

To determine whether instructional explicitness or surface format explains part of the representation effect.

3.10.2 Procedure

Use the same 30 puzzles on all models and vary one factor at a time:

Rules: minimal | explicit constraints | constraints+alphabet | fully explicit

Mapping: alphabet only | $\alpha \leftrightarrow 1, \dots, 9$ | $\alpha \leftrightarrow V1, \dots, V9$

Output: spaced rows | compact rows | one 81-symbol string | JSON

Empty: . | 0 | _ | EMPTY

Case and spacing variants are included for Latin and nonce labels. The main protocol is unchanged by later ablation results.

3.10.3 Example Solution

The following illustrates JSON structure; the real answer contains nine rows:

```
[["1","2","3","4","5","6","7","8","9"],  
 ["4","5","6","7","8","9","1","2","3"]]
```

3.11 Experiment 10

3.11.1 Test

To test whether a model can find and fix errors in its own answer. The experiment also compares the model's self-check with an automatic Sudoku checker.

3.11.2 Procedure

Use the same 30 puzzles in Arabic, Greek, and emoji form. For each model and form, test every puzzle under the following four conditions. Start a new conversation for each condition. This prevents an answer or feedback from one condition from affecting another.

1. **One pass (baseline):** ask the model to solve the puzzle once. Do not ask it to check or change its answer.
2. **One self-revision:** ask the model for an answer. Then ask it to check the answer and return a revised answer. The model must check that the answer has 81 cells, keeps all given clues unchanged, and follows the rules for every row, column, and 3×3 box.

3. **Two self-revisions:** ask the model for an answer and give it two chances to check and revise the answer. This condition has three answers: the initial answer, the first revision, and the final revision.
4. **Checker-guided revision:** check the initial answer with the automatic Sudoku checker. Send the error messages to the model and allow one revision. If the initial answer has no errors, record it as correct and do not ask for a revision.

The automatic checker tests whether the answer can be read, has 81 cells, keeps all given clues unchanged, and follows every row, column, and box rule. It describes each error but does not give the correct value. For example:

Row 4 contains two ε symbols.
 Column 7 is missing θ .
 Given cell r2c4 changed from α to β .

Save and check every answer, including each revised answer, with the automatic checker. For each condition, report the accuracy of the first and final answers, the number of wrong answers that the model fixed, and the number of correct answers that became wrong after revision. Also report the number of model calls, token use, and total time. Do not treat the model’s claim that an answer is correct as proof. Always use the automatic checker.

4 Bibliography and References

- [1] R. Pelánek, “Difficulty rating of Sudoku puzzles: An overview and evaluation,” *arXiv:1403.7373*, 2014. <https://doi.org/10.48550/arXiv.1403.7373>
- [2] J. Seely, Y. Imajuku, T. Zhao, E. Cetin, and L. Jones, “Sudoku-Bench: Evaluating creative reasoning with Sudoku variants,” *arXiv:2505.16135*, 2025. <https://doi.org/10.48550/arXiv.2505.16135>
- [3] K. Shah, N. Dikkala, X. Wang, and R. Panigrahy, “Causal language modeling can elicit search and reasoning capabilities on logic puzzles,” *Advances in Neural Information Processing Systems*, vol. 37, 2024.
- [4] K. Stechly, M. Marquez, and S. Kambhampati, “GPT-4 doesn’t know it’s wrong: An analysis of iterative prompting for reasoning problems,” *NeurIPS Foundation Models for Decision Making Workshop*, 2023. <https://doi.org/10.48550/arXiv.2310.12397>
- [5] A. T. Wasi *et al.*, “InvariantBench: Can large language models exhibit inherent reasoning across equivalent transformations?” *3rd AI for Math Workshop at ICML*, 2026.
- [6] I. Mirzadeh *et al.*, “GSM-Symbolic: Understanding the limitations of mathematical reasoning in large language models,” *ICLR*, 2025.
- [7] J. Wei *et al.*, “Symbol tuning improves in-context learning in language models,” *EMNLP*, pp. 968–979, 2023. <https://doi.org/10.18653/v1/2023.emnlp-main.61>
- [8] K. Bostrom and G. Durrett, “Byte pair encoding is suboptimal for language model pretraining,” *Findings of EMNLP*, pp. 4617–4624, 2020. <https://doi.org/10.18653/v1/2020.findings-emnlp.414>
- [9] P. Rust, J. Pfeiffer, I. Vulić, S. Ruder, and I. Gurevych, “How good is your tokenizer? On the monolingual performance of multilingual language models,” *ACL-IJCNLP*, pp. 3118–3135, 2021. <https://doi.org/10.18653/v1/2021.acl-long.243>
- [10] Z. Zhou, M. Li, X. Fang, X. Zhou, W. Lin, and Z. Zheng, “LGMT: Logic-grounded metamorphic testing for evaluating the reasoning reliability of LLMs,” *Knowledge-Based Systems*,

vol. 348, art. 116324, 2026.

- [11] Y. Yan, Y. Lu, R. Xu, and Z. Lan, “Do PhD-level LLMs truly grasp elementary addition? Probing rule learning vs. memorization in large language models,” *arXiv:2504.05262*, 2025. <https://doi.org/10.48550/arXiv.2504.05262>
- [12] Qwen Team, “Qwen3.8-27B model card,” 2026. <https://huggingface.co/Qwen/Qwen3.8-27B>
- [13] GLM-5 Team, “GLM-5.2 model card,” 2026. <https://huggingface.co/zai-org/GLM-5.2>
- [14] Moonshot AI, “Kimi-K3 model card,” 2026. <https://huggingface.co/moonshotai/Kimi-K3>
- [15] OpenAI, “Models: GPT-5.6 Terra,” 2026. <https://developers.openai.com/api/docs/models>
- [16] Anthropic, “Models overview: Claude Sonnet 5,” 2026. <https://platform.claude.com/docs/en/about-claude/models/overview>

Signature of Student

Signature of Supervisor